Serialization Formats

JSON, MessagePack, Protobuf and Avro, and evolving schemas safely

Watch the 4-min video

In one line: JSON is readable and universal but verbose and slow; MessagePack is compact schemaless binary JSON; Protobuf and Avro use schemas, so messages are small and fast and schemas evolve under clear rules; use JSON for public APIs, Protobuf for gRPC, Avro with a schema registry for event streams, and never pickle across services.

1 JSON at a hundred thousand events a second

Your services exchange events as JSON. It's readable and easy. But at 100,000 events per second, JSON parsing burns CPU, every message repeats the field names, and one team renames a field and breaks three consumers. The serialization format matters more than it looks.

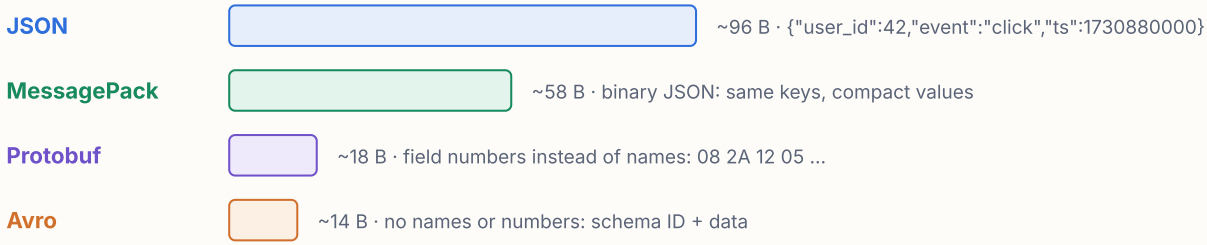


Figure 1. One small event in four formats (approximate sizes): schemas let binary formats drop the field names.

2 JSON

The default: human-readable, supported everywhere, no schema needed. But it's verbose (field names repeat in every message), numbers are text, binary data needs base64, and parsing is relatively slow. In Python, **orjson** is several times faster than the standard `json` module. JSON is still right when humans or browsers read the data.

3 MessagePack

Binary JSON: the same data model, no schema, but more compact and faster to parse. A good fit for caches (lesson 33) and internal messages when you want speed without defining schemas. Libraries exist for almost every language.

4 Protocol Buffers

A schema in a `.proto` file. Fields are identified by **numbers**, not names, so messages are small and parsing is fast. Code is generated for each language, and Protobuf powers gRPC. The evolution rules are simple: add new fields with new numbers, never reuse a number (mark removed ones `reserved`), and old and new code keep working together, because readers skip fields they don't know (Phase 2, API lesson 33).

```
syntax = "proto3";
message ClickEvent {
  int64  user_id = 1;
  string event   = 2;
  int64  ts       = 3;
  string page    = 4;   // added later: old readers skip it
  reserved 5;          // removed field: number never reused
}
```

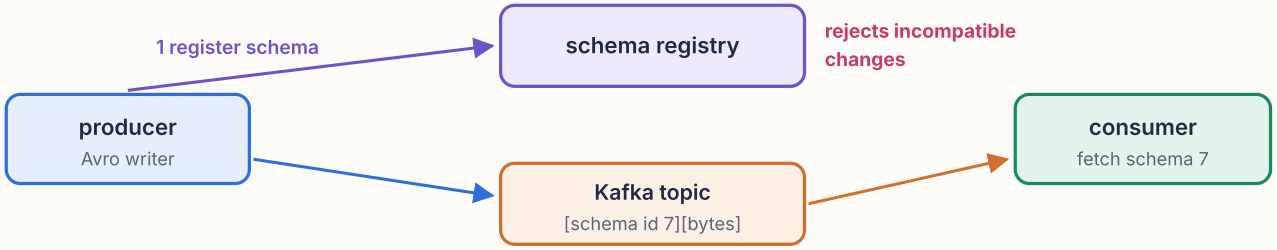
Figure 2. A Protobuf schema that evolved safely: a new field with a new number, and a removed number reserved.

5 Avro and a schema registry

Avro also uses a schema, but the schema is stored **separately**, often in a **schema registry**. Messages carry no field names or numbers at all, just the data. The reader resolves the **writer's** schema against its own **reader** schema, which makes evolution well defined; **default values** let you add a field without breaking old readers. Avro is the classic choice for Kafka events and data lakes.

WORKED EXAMPLE: EVENTS FROM JSON TO AVRO

Producers register the event schema in a schema registry (Confluent, Apicurio). Each message carries a small schema ID plus binary data. Messages shrink by more than half. The registry enforces a compatibility mode (for example BACKWARD), so a breaking change, like removing a field without a default, is **rejected before it reaches production**.



each message carries a 5-byte header (magic + schema ID) instead of field names

Figure 3. A schema registry: producers register schemas, messages carry only a schema ID, consumers fetch the schema to decode.

6 How to choose

Use case	Format
Public APIs, browsers, debugging	**JSON** (orjson in Python)
gRPC and service-to-service calls	**Protobuf**
Event streams, data pipelines	**Avro** (or Protobuf) with a schema registry
Compact, schemaless internal data, caches	**MessagePack**
Across services	never **pickle**: Python-only and unsafe with untrusted input

Whatever you pick, version your schemas and test compatibility in CI.

COMMON MISTAKES

- Reusing a Protobuf field number for a different field.
- Renaming JSON fields in an event without versioning, breaking consumers silently.
- Adding an Avro field without a default, breaking old readers.
- Large integers in JSON losing precision in JavaScript (above 2^53).

7 Key terms

Serialization: turning objects into bytes, and back

Field number: Protobuf's on-the-wire identity for a field

Writer / reader schema: Avro resolves the two when decoding

Schema: a formal description of the fields and types

Schema registry: stores schemas and checks compatibility

Compatibility mode: rules such as BACKWARD or FULL

8 Interview questions

Q1. Which serialization format would you use for events and why? BASIC

Avro or Protobuf with a schema registry, because they're compact and fast, and they enforce compatible schema evolution between producers and consumers.

Q2. Why not JSON everywhere? BASIC

It's verbose and slower to parse, and without a schema a renamed field can silently break consumers.

Q3. How do you add a field to a Protobuf message used by 20 services? SCENARIO

Add it with a new field number (optional), deploy readers and writers in any order since unknown fields are skipped, and never reuse or renumber existing fields.

Q4. A team wants to remove a field from an Avro event. How? SCENARIO

Under backward compatibility, readers must cope: give the field a default in the reader schema first, then stop writing it; the registry blocks removals that would break existing consumers.

Q5. Why does Avro need the writer's schema to decode? TRICKY

Messages contain only values in field order, no names or numbers, so the reader must know exactly which schema wrote them, which the schema ID in the message points to.

Q6. Why is pickle dangerous between services? TRICKY

Unpickling can execute arbitrary code embedded in the data, and it ties every consumer to Python and compatible class definitions.

Next: SD-J-009 · Resource Utilization (USE Method) backendroadmap.com